



Why Enterprise AI Projects Fail Beyond The Demo

And how enforceable architecture standards come to your rescue.

DHEERAJ SAXENA

FOUNDER@DATAWHISTL.COM

ABOUT

Founder @ Datawhistl.com

Data and AI Consultant advising mid-market and enterprise clients across CRM and Marketing domains.

5+ yrs in Enterprise AI Architecture and Advisory across UK, EU

AI Enterprise Architecture, Operating Model Design, AI & Data Readiness Assessments.

27+ yrs of Enterprise Data Consulting Heritage

Data Strategy & Architecture Leadership for multi-million-dollar enterprise data platforms.

CONTACT ME



+44-7950741519



LINKEDIN.COM/IN/DHEERAJSAXENA



DHEERAJS@DATAWHISTL.COM



Before We Dive In



45 minutes

35 talk · 10 Q&A



Intro session

overview, not deep-dive



Technical

not hands-on



Questions

at the end / in chat

TODAY · FOUR SECTIONS

1

**Setting the
context**

the enterprise problem

2

**Review a failure
scenario**

1 real production scenario

3

**What is
the fix**

enforceable standards

4

**Cohort agenda
overview**

what the workshop covers

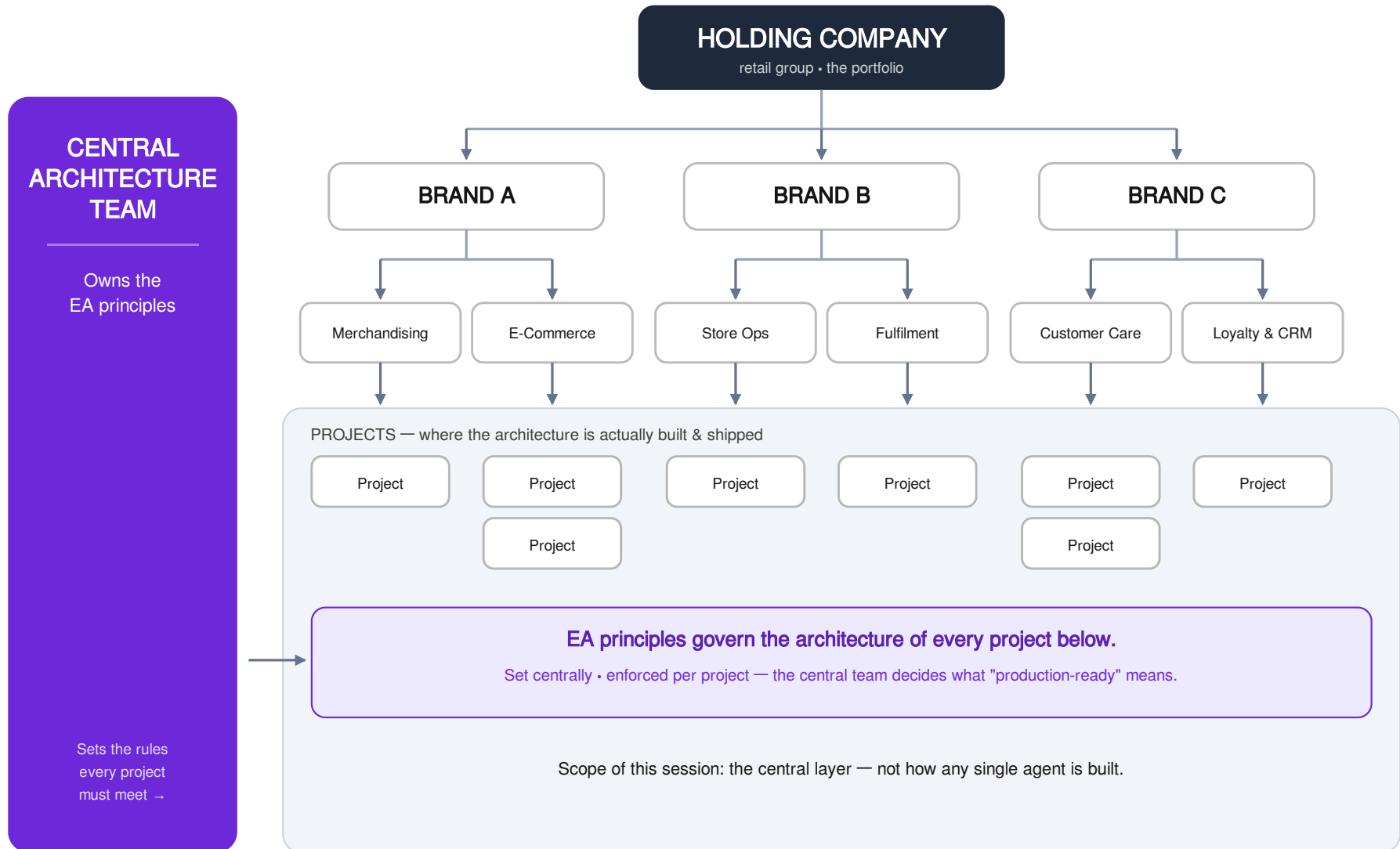
Focus: HOW to develop AI Architecture Standards — not why.

Developers welcome — best suited to Senior Architects, IT Ops Managers & AI Delivery Leads.

Section 1

THE ENTERPRISE CONTEXT

The Enterprise Context



Reliable Project by Project

Most teams climb from stage one to stage two — and assume two is the destination.

01

STRUGGLING

**Works in Dev,
Fails in Production**

● **Breaks in production**

Demos impress — then break under real load, data and users. Readiness is an afterthought.



02

MATURING

**Optimized
Project by Project**

● **Reliable — one project at a time**

Properly engineered — evals, guardrails, monitoring. But the discipline is reinvented team by team.

Every project is reliable. So where's the enterprise problem?

Project-level excellence still fails the enterprise

Five things break at portfolio scale — only central control holds them.



Projects optimise for launch — only central control survives all five.

COST

- What's at stake:** Total spend across the portfolio — token/inference, hosting, and duplicated build effort — not any single project's line item.

Projects On Their Own

- Each team optimises its own cost in isolation, or not at all.
- An uncached prompt prefix, model/task mismatch — each looks trivial inside one project but invisible across thirty.
- Waste grows linearly with traffic and nobody is on the hook for it.

Central

- One standard mandates the cost controls — prompt caching, right-sized models, usage caps — and emits per-project cost attribution.
- The aggregate has an owner and gets reviewed across the portfolio.
- Waste is caught at PR time, not discovered on the invoice.

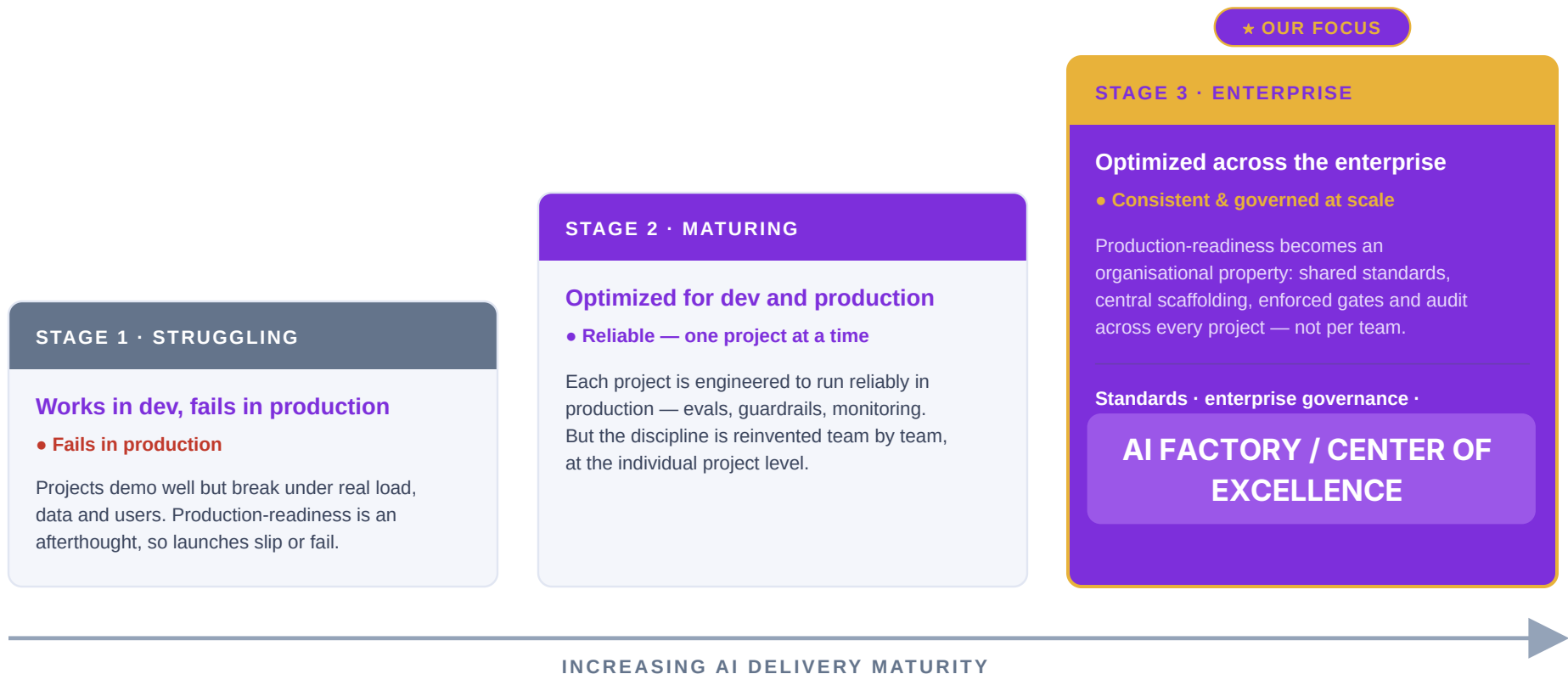
Full set of examples

Engineering AI to Survive Production

Enterprise Perspective

Three stages of AI delivery maturity

From projects that break in production to delivery governed across the entire enterprise.



Section 2

REVIEW FAILURE SCENARIO

Enterprise AI — Non-Functional Failure Modes

The cross-cutting concerns projects don't test for themselves

LEGAL & COMPLIANCE

- GDPR — data privacy & retention
- EU AI Act & sector regs (FCA, HIPAA)
- Auditability & evidence trail
- IP & copyright exposure

ENTERPRISE AI Failure Modes non-functional, cross-cutting

CUSTOMER EXPERIENCE

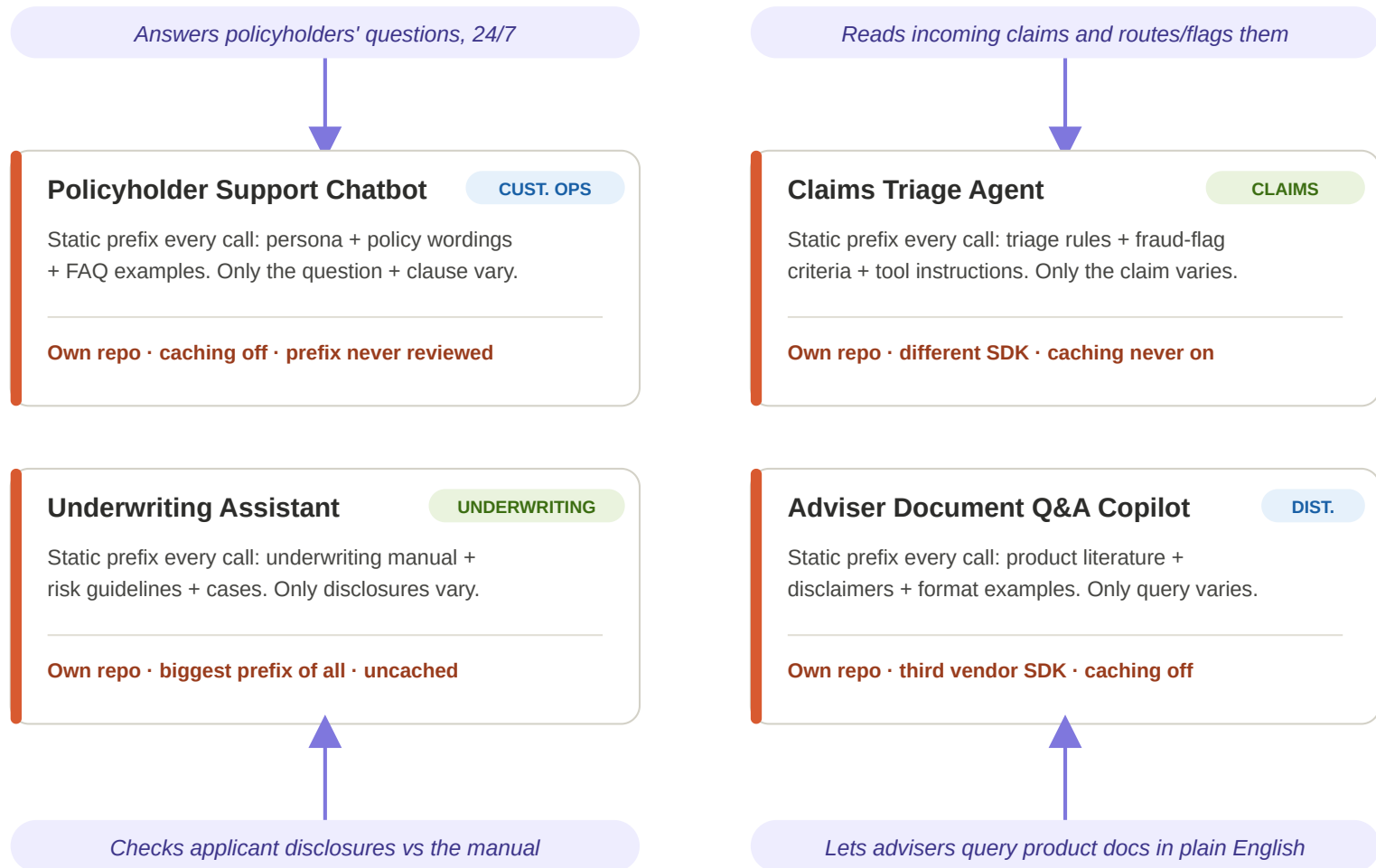
- Accuracy & hallucination
- Latency & responsiveness
- Consistency / model drift
- Safety & brand trust

COST

- Capex — platform & build investment
- Opex — token / inference & hosting
- Engineering toil & duplication
- Incident & remediation cost

Review a failure

Not one bot — four projects, four departments at Phoenix Life, each calling LLMs its own way.



The same uncached-prefix waste — built four separate times.

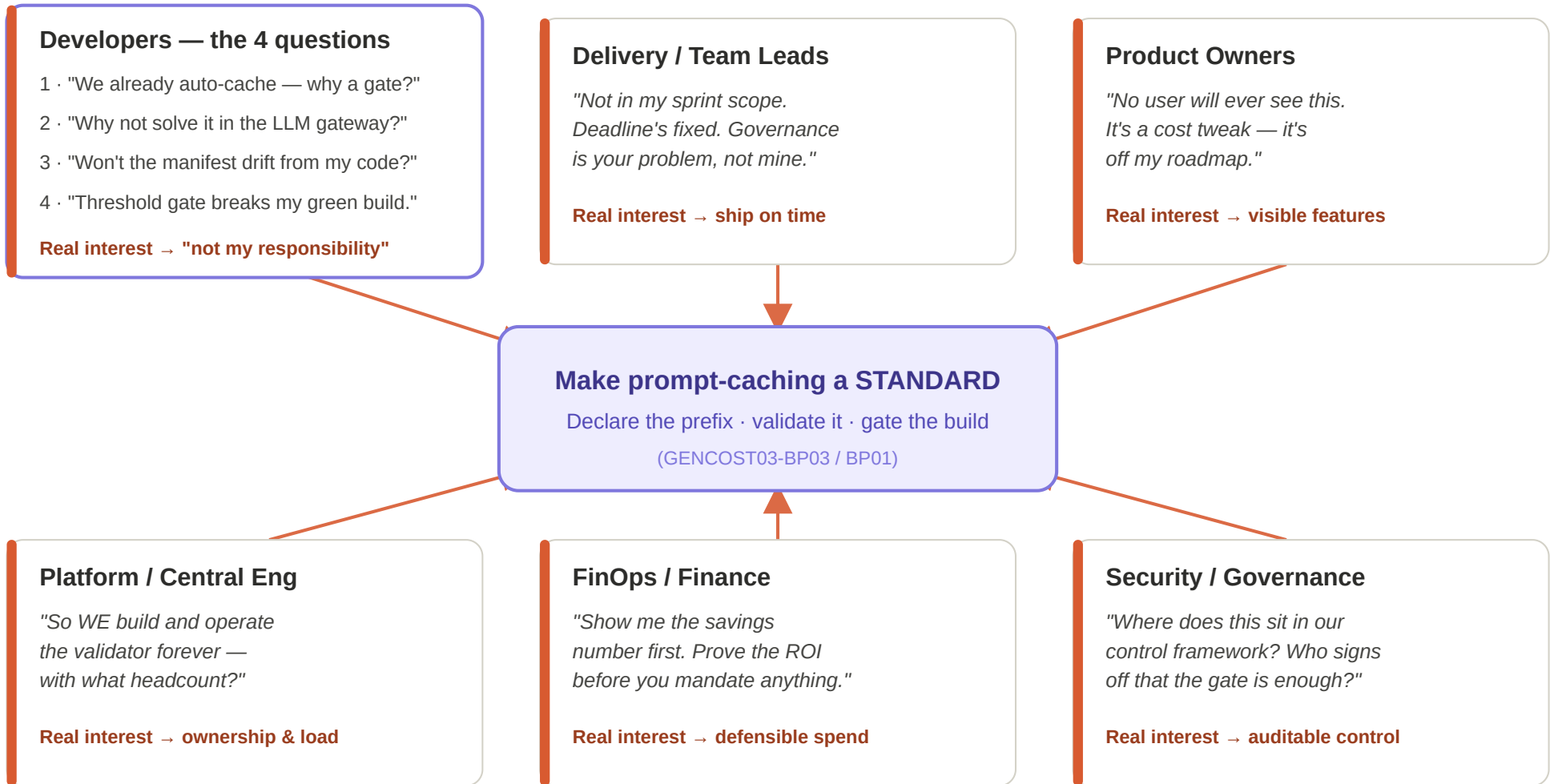
Four departments. Four codebases. Three different SDKs. Zero shared standard.

No one place enforces it. No one place can even see it — spend lands as four diffuse "model spend" lines.

The failure isn't the bot. It's that "how we call an LLM" was never standardized.

Turn the fix into a standard — and the room pushes back

Same proposal, six sets of incentives. Every arrow is a "no."



The GUI demo was never the hard part.

The real gap in enterprise AI is getting six teams with conflicting incentives to adopt and enforce one standard.

That negotiation — not the prototype — is where Enterprise Architecture earns its keep.

Section 3

THE FIX?

The FIX Needed

Two parts. Both required.

Part 1 — Enterprise-Wide AI Architecture Standards: *With Reference Implementations*

- A catalog of standards with ***reference implementations*** (not just best practice statements)
- Objective basis for ARB, architects, and engineers to make and defend decisions.

Part 2 — Governance to Enforce Them

- Early involvement of AI Enterprise Architects.
- Pre-project architecture approvals.
- Milestone reviews in ARB. Pre-commit verifications before production deployment.
- ***Develop/treat standards as products***

Cohort Workshop focus is **Part 1**

Making it a standard — who owns what

Central builds the machine. Projects wire into it. They meet at one contract.

Central / Platform team

BUILDS · OPERATES · OWNS

The validator

render-twice-and-diff the prefix

The threshold table

per-model min cache size + tokenizer, versioned

The breakpoint emitter

inserts the cache marker at runtime / gateway

The endpoint contract

defines `build_prompt(inputs) → blocks`

The CI gate

the check projects plug into; red = blocked

Build it once, for everyone.

INTERFACE CONTRACT

`build_prompt()`

Project / Delivery team

CONFIGURES · POPULATES · CONSUMES

One endpoint

route every prompt through `build_prompt()`

Mark the breakpoint

declare what's prefix vs suffix

Keep variables below the line

nothing dynamic in the static prefix

Run the check in CI

call the central gate; fix on red

Don't reinvent caching

consume the validator, don't hand-roll it

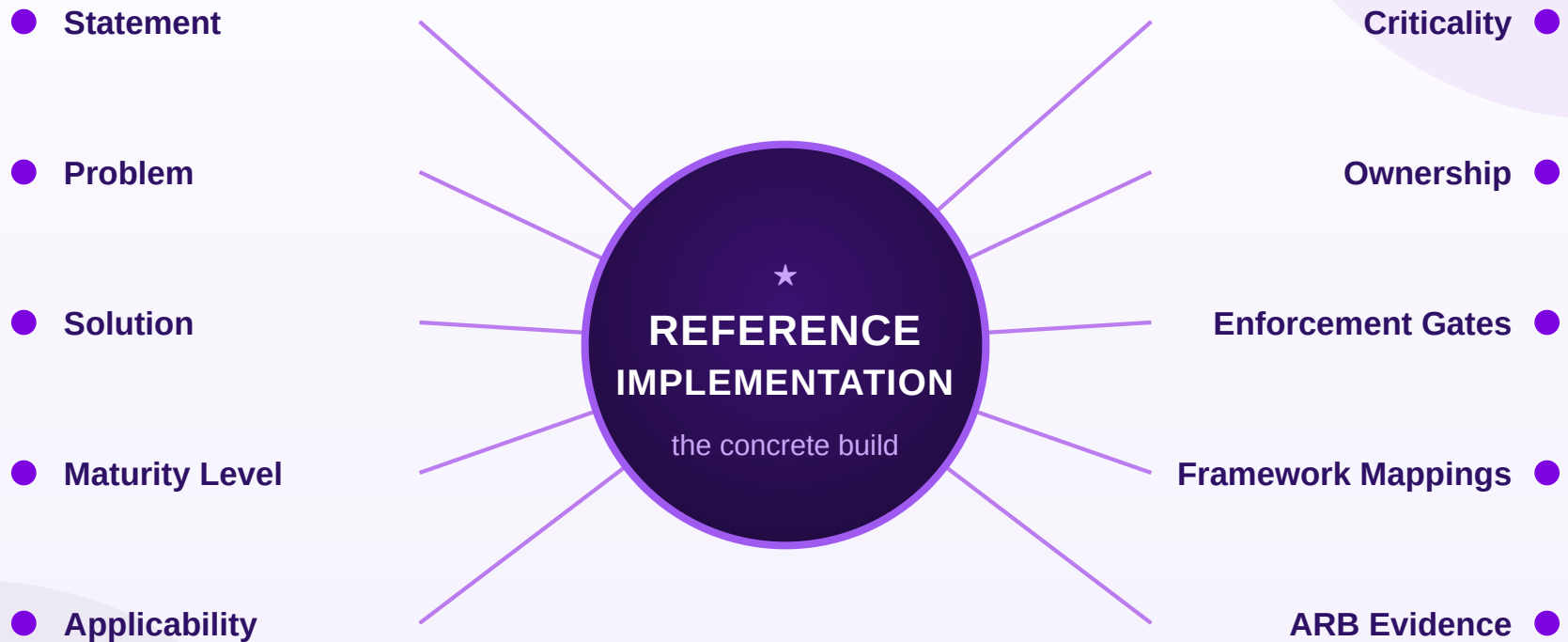
Wire into it. Don't rebuild it.

The contract is the standard — not a memo asking everyone to "remember to cache."

Every prompt flows through one declared endpoint, so the gate can prove caching works instead of trusting it does.

What a Standard Actually Looks Like

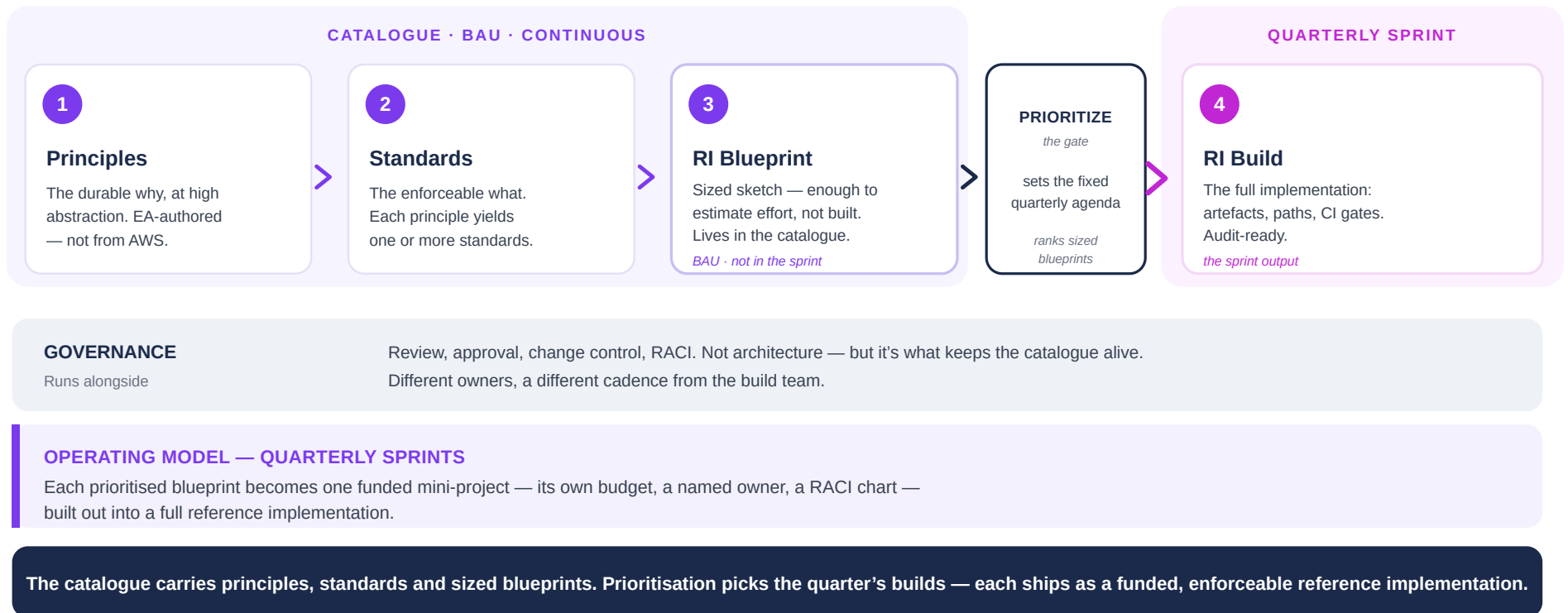
A taxonomy of nodes — anchored by a concrete reference implementation.



Concrete Reference Implementation — enforced at the ARB & CI gates.

How the EA Team Builds the Catalogue

Principles → Standards → RI Blueprint → Prioritize → RI Build. The catalogue runs continuously; builds run as quarterly sprints.



What the Central EA Team Does

One function, three jobs — set centrally, enforced everywhere.

01



IDENTIFY

Identify the standards

Define what "production-ready" means — the cross-cutting concerns no single project owns or tests.

02



PRIORITIZE

Prioritize what to build

Rank candidates by risk and value — the gate that sets each quarter's build agenda.

03



DEVELOP

Reference implementations

Turn each standard into a concrete, reusable blueprint — gated in CI, audit-ready.



Set centrally · enforced per project — the central team decides what "production-ready" means.

IDENTIFYING THE BASELINE

Where Do the Baseline Standards Come From?

No single source covers all three dimensions — frameworks lean technical, regulation covers legal. You compose.

COST

spend vs. value

BESPOKE

first-principles

No external framework
fits — you build the
cost model yourself.

CUSTOMER EXP.

trust in the product

BESPOKE

first-principles

No external framework
fits — you define the
CX heuristics yourself.

LEGAL & RISK

what you're liable for · many sources

★ EU AI Act

GDPR

FCA

NIST AI RMF

★ AWS GenAI Lens

Internal Risk / Compliance

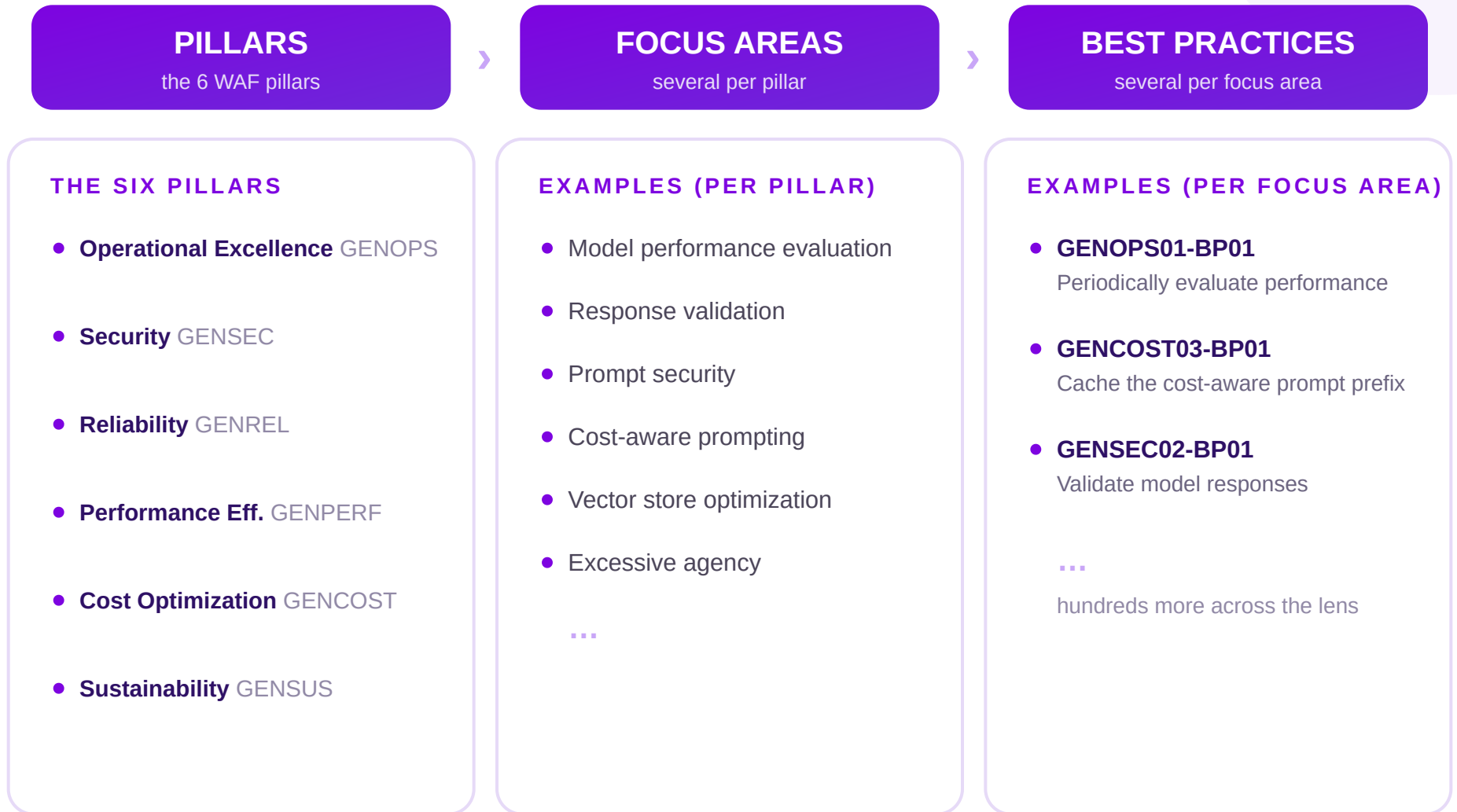
THIS WORKSHOP DEMONSTRATES ONE METHOD — IN THE LEGAL CATEGORY

★ EU AI Act → obligations

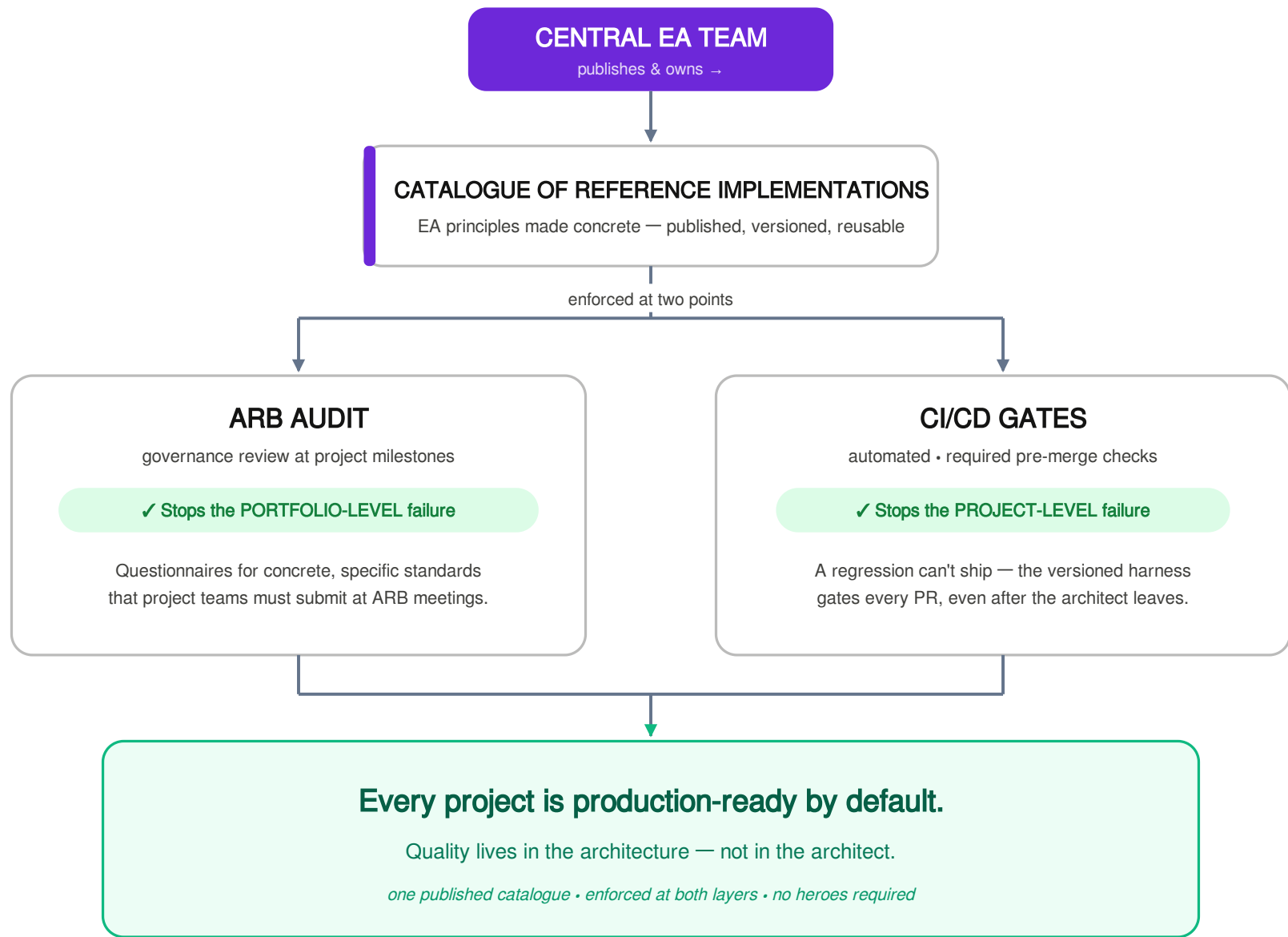
★ GenAI Lens → technical build

How the GenAI Lens Is Structured

A drill-down: each pillar holds focus areas; each focus area holds best practices.



We baseline our catalogue on the GenAI Lens — selected best practices become enforceable standards.



Section 4

COHORT AGENDA

Today was the process intro

The build happens in the full workshop

5 parts

1

Frame the problem

Deep-dive into 3 failure scenarios

2

Develop the baseline catalog

Establish the starting set of standards

3

Build standards as products

The full end-to-end process

4

Hands-on session

5

Bonus sections

Aegis intro · Operating model · Product strategy

Bonus

July 2 Workshop – What you get

Part 1 — The Problem

What counts as a production failure?

Before you can prevent failure, you have to agree on what it is. It falls into three categories.

Legal & Risk

Harm the business is liable for

- PII & sensitive-data leakage
- Regulatory non-compliance
- Unauthorized data access
- Harmful or prohibited output
- No audit trail or traceability

Customer Experience

Erosion of trust in the product

- Accuracy & quality drift
- Hallucinated or fabricated answers
- Inconsistent, off-brand tone
- Failed task completion
- Slow or unreliable responses

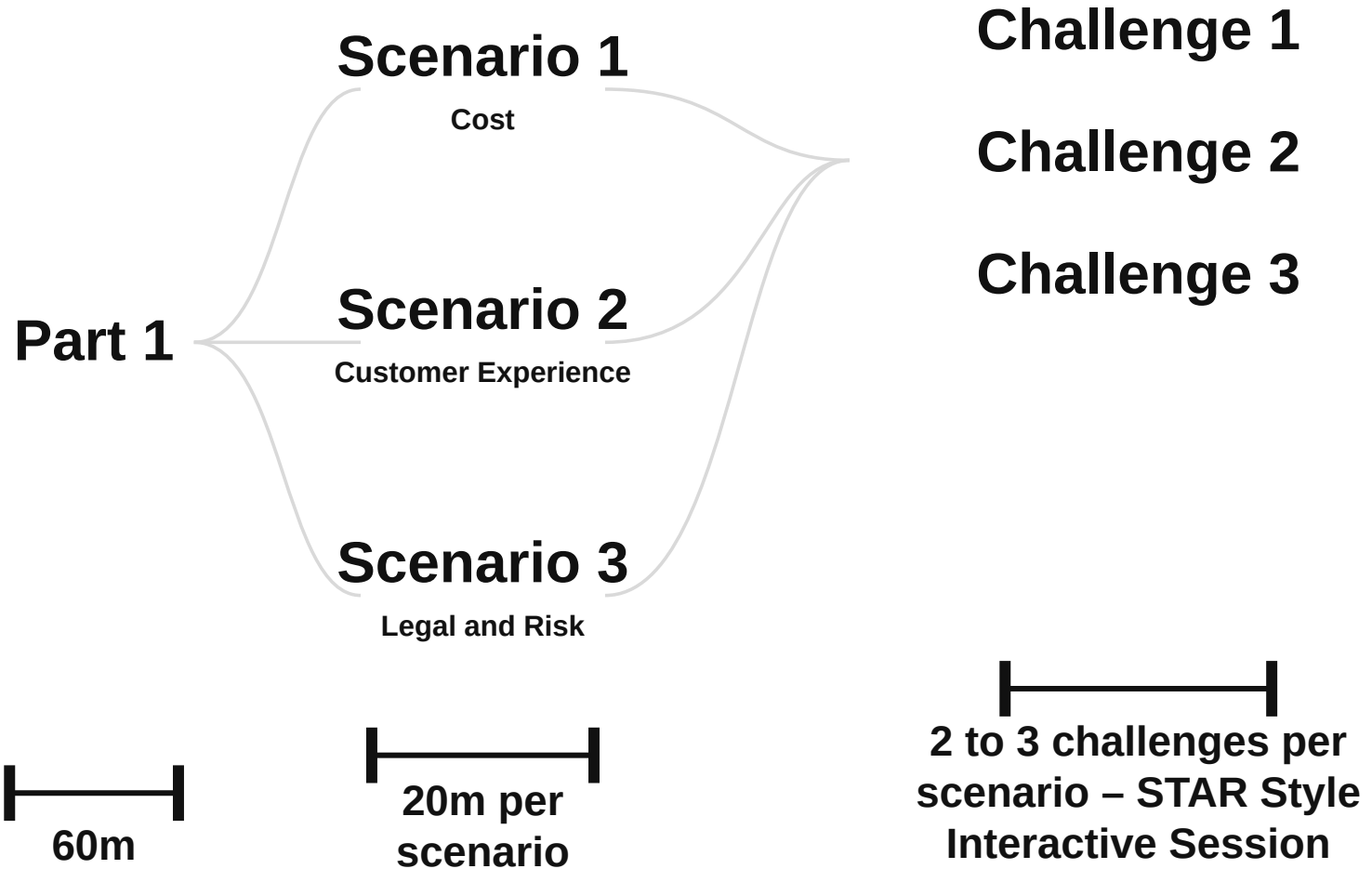
Cost

Spend that outruns the value

- Oversized-model token burn
- Runaway or unbounded usage
- Context & retrieval bloat
- Over-provisioned infrastructure
- No cost visibility or attribution

Three examples. Passing development isn't passing production —
and passing production still isn't success for the company. That third gap is why standards exist.

July 2 Workshop – What you get



July 2 Workshop – What you get

Part 2 — Building the Catalogue

Where do the standards come from? Choosing what to baseline against.

1 Every standard serves one of three dimensions

Legal & Risk

what the business is liable for

Customer Experience

trust in the product

Cost

spend vs. value

2 No single source — you choose what to baseline against

Frameworks

AWS GenAI Lens,
NIST AI RMF,
ISO 42001

Regulation

EU AI Act, GDPR,
sector rules

Incidents

Post-mortems,
near-misses

Risk model

Threat modeling,
risk register

Bespoke

First-principles
(cost, CX)

No source covers all three dimensions — frameworks lean operational, regulation covers legal, cost & CX are usually bespoke. You compose.

3 A closer look at the framework we use: the AWS lens family

ML Lens

Full ML lifecycle — broadest scope

GenAI Lens

LLM · RAG · agentic — speaks our language

Responsible AI Lens

Fairness, safety, privacy — 8 focus areas

Well-Architected Framework — the foundation

Operational Excellence · Security · Reliability · Performance Efficiency · Cost Optimization · Sustainability

A lens re-applies the six pillars to one kind of workload. We baseline the catalogue on the GenAI Lens.

July 2 Workshop – What you get

Part 2-How a candidate becomes a standard

— PART 2 · BUILDING THE CATALOGUE

How a Candidate Becomes a Standard

1

Choose the Anchor

(which baselines we take — regulation, frameworks, or bespoke)

2

Keep or Drop

(GenAI-distinctive? already covered? enforceable? material?)

3

Score the Survivors

(criticality · difficulty · reusability · enforcement cost)

4

Write the Standard

(statement + gate + evidence + applies_to)

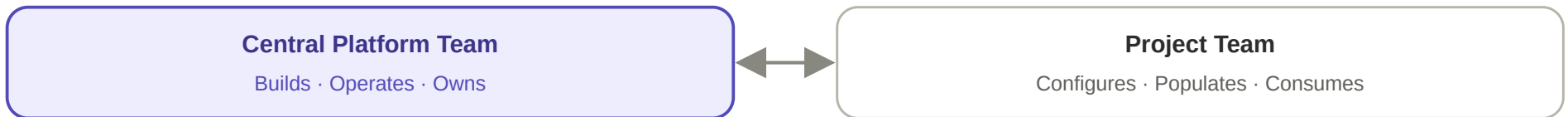
July 2 Workshop – What you get

Part 3 — The Process

Turning a candidate standard into something a team can actually build.



Then: who builds it, and who owns it?



The gate guarantees a control exists; the project owns its quality; governance owns the consequences.



We'll also touch on the operating-model org chart

Head of AI · Lead Architect · Head of Engineering · Head of Product · Head of Governance

How they all connect — that reveal is live in the session.

July 2 Workshop – What you get

Part 4 — Hands-On Lab

Walk the process for real — three standards, you build one.

1 Three standards — one per dimension

Legal & Risk

candidate standard

Customer Experience

candidate standard

Cost

candidate standard

We walk all three through Blueprint → Prioritize → Concrete RI. You take one the whole way.

2 Build one from a sample taxonomy — the anatomy of a standard

We hand you a sample taxonomy and you fill in every field to develop the concrete build.

Statement

Problem

Solution

Maturity Level

Applicability

Criticality

Ownership

Enforcement Gates

Change Control

Framework Mappings

ARB Evidence

Reference Impl.



WHAT YOU WALK AWAY WITH

Concrete Reference Implementation

OWNED BY
Central AI Factory

BUILT BY
AI Engineering Platform Team

MAINTAINED BY
Central AI Product Team

✓ Automatically enforced at CI/CD gates

✓ Fully auditable

July 2 Workshop – What you get

Part 5

Repository of Enterprise AI Architecture Standards - Fully accessible via Claude MCP Connector

 Datawhistle

Refresh

Dashboard

All Principles

All Standards

Taxonomy

Not promoted

SEARCH

Search standards...

PILLAR

All pillars

FOCUS AREA

All focus areas

MATURITY LEVEL

All maturity levels

AWS BEST PRACTICE

All best practices

NIST CATEGORY

All NIST categories

NIST SUBCATEGORY

All NIST subcategories

All Standards

ST-G01B1-01

Operational Excellence

Prove an agent still behaves before you ship a change to it

A change to an agent should not reach production until it has been checked against known-good behaviour. The team should be...

Model Performance Evaluation

ST-G01B1-02

Operational Excellence

Judge an agent on its weakest group, not its average

Aggregate quality hides failures in specific slices of users or cases. An agent's quality should be judged per meaningful group, so ...

Model Performance Evaluation

ST-G01B1-03

Operational Excellence

Measure quality the same way every time

How an agent's quality is scored should be stable, inspectable, and testable on its own. When scoring is redefined ad hoc each run, ...

Model Performance Evaluation

ST-G01B1-04

Operational Excellence

Notice a model drifting before your users do

A model that passed at deployment can quietly degrade as its inputs and the world around it change. Someone should be...

Model Performance Evaluation

ST-G01B1-05

Operational Excellence

Keep the yardstick current with how the system is really used

An evaluation set frozen at last year's usage gives false confidence. It should be kept current on a known cadence, so it keeps...

Model Performance Evaluation

ST-G01B1-06

Operational Excellence

Know which model you're running, and re-prove behaviour before you change it

A model can change underneath a workload without warning, through provider roll-forwards, deprecations, or default upgrades...

Model Performance Evaluation

ST-G03B1-01

Operational Excellence

Treat prompts as governed assets, not scattered strings

Prompts buried inline across the codebase cannot be tracked, reviewed, or rolled back. They should be managed as deliberate,...

Traceability

ST-G03B2-01

Operational Excellence

Make every AI workload observable through one consistent channel

When each workload emits telemetry its own way, nothing can be debugged or audited across the estate. AI observability should flo...

Traceability

ST-G03B2-02

Operational Excellence

Traces can hold sensitive data, so govern who reads them and how long they live

Prompts and responses captured in observability traces may contain personal or confidential data. Who can read those trace...

Traceability

ST-GC1B1-01

Cost Optimization

Choose a model deliberately, and record why

Model choice should be a reasoned decision against stated requirements and cost, not a default taken unquestioned. The reasoning...

Model Selection & Right-sizing

ST-GC2B2-01

Cost Optimization

Size AI infrastructure to real demand, not to the safe maximum

Over-provisioned hosting quietly wastes large amounts of spend, because the safe default is almost always bigger than the workload...

Inference Cost Optimization

ST-GC3B1-01

Cost Optimization

Keep the cost of every prompt under deliberate control

Prompts grow silently as instructions are appended over time, inflating per-call cost with no natural review moment. Each prompt...

Cost-aware Prompting

Currently GenAI focused, rapidly evolving to include ML, and wider AWS WAF Standards

July 2 Workshop – What you get

Part 5

— IN THE FULL WORKSHOP

Operating Model & Product Strategy

A preview of what the paid workshop goes deep on.

OPERATING MODEL

how the AI function is organised

- ✓ **AI CoE structure**
the shape of the central team
- ✓ **EA Function**
where AI sits within EA
- ✓ **Engineering**
builds & ships the standards
- ✓ **Product**
runs SLA & lifecycle
- ✓ **Governance**
independent certification
- ✓ **Interplay with wider EA**
how it connects to the rest of EA

PRODUCT STRATEGY

how enforcement matures over time

1

Phase 1 — Self-attestation

Projects certify themselves against the standard.



2

Phase 2 — EA spot-checks

Central EA audits projects at random — trust, but verify.



3

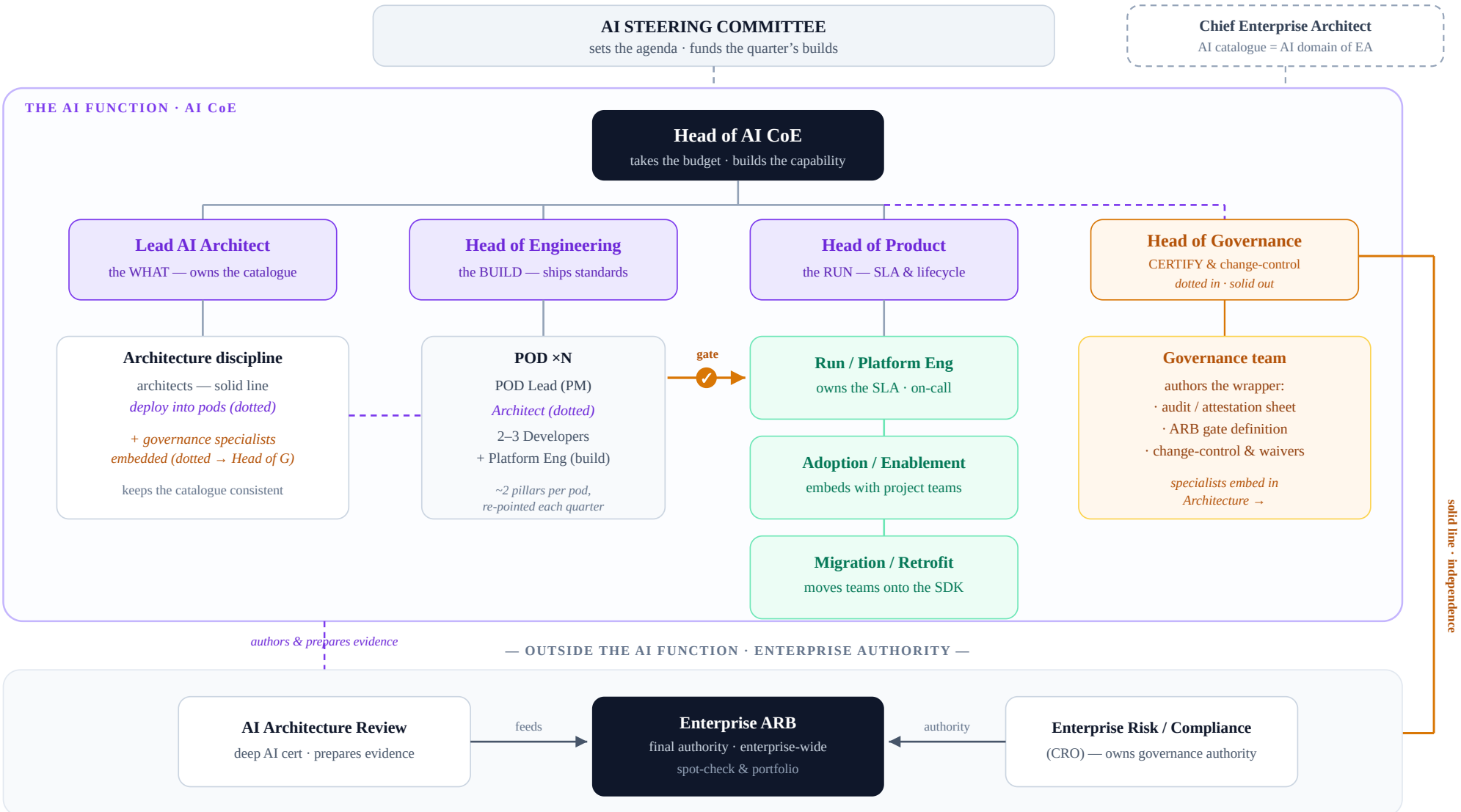
Phase 3 — Central SDK

Enforcement automated — baked into a central SDK.

The org that runs it · the path from attestation to automation.

July 2 Workshop – What you get

Part 5



July 2 Workshop – What you get

Part 5

The Enforcement Journey — Attest → Verify → Provide

Each rung is stronger, costlier, and harder to commit to. The control only gets real as you climb — and most never leave the bottom.

LEVEL 1 ATTEST



Project self-certifies.
Proves it was claimed —
not that it works.

EFFORT: LOW
Enforceable today, no tooling

WHAT IT IS

A guardrail YAML per workload: every threat dispositioned (mitigated / accepted / N-A), named owner, signed at the ARB.

Gate: every threat dispositioned + signed (completeness check — manual, or a 20-line CI linter). Not efficacy.

EA · Architect

Owens the threat taxonomy + standards; sets applicability metadata that filters the list.

EA · Governance

Runs the ARB review; challenges the N-A / accepted dispositions; certifies; wires to post-incident.

Engineering

Minimal — optionally a completeness linter for the YAML.
No platform needed.

Product

Owens the attestation template + submission process; tracks which workloads are signed.

Project

Authors the YAML, dispositions every relevant standard, names the owner, signs.

⚠ Most enterprises stop here. Commitment ends at the cheapest rung — and that is why AI fails beyond the demo.

LEVEL 2 VERIFY



EA attacks the running system. Proves it works — claim replaced by proof.

EFFORT: HIGH
Needs a central platform

WHAT IT IS

EA fires an adversarial corpus at the deployed endpoint and scores the responses. Project can't fake it — only probe-testable controls qualify.

Gate: 0 successful overrides on the central corpus + false-positive rate under threshold.

EA · Architect

Defines "pass": threshold (0), what counts as an override, which controls are probe-testable.

EA · Governance

Owens the verdict + gate. Consumes the report, certifies, blocks release.
Independent.

Engineering

BUILDS the harness, corpus library + generator, scoring / oracle engine; keeps the corpus current.

Product

RUNS it as a service: schedules runs, SLA, cost pool, report, tracks verification coverage.

Project

Staging endpoint + creds, the invariants / oracle, a ~15-line adapter, fixes fails.

LEVEL 3 PROVIDE



EA owns the control as non-bypassable infra. Compliance = "routed?"

EFFORT: HIGHEST
Platform + egress control

WHAT IT IS

A central SDK / gateway does the filtering, scanning, tool-auth, logging. SDK alone is NOT L3 — only SDK + enforced egress routing.

Gate: egress/IAM policy blocks every direct call to the model — routing proof + signed attestation + build provenance.

EA · Architect

Owens what the SDK / gateway must enforce and the routing policy's required behaviour.

EA · Governance

Gate = routing proof + provenance + signed attestation. Certifies; owns waivers for genuine gaps.

Engineering

BUILDS + maintains the SDK, gateway, egress / IAM enforcement, filtering, migration tooling.

Product

OPERATES the gateway: SLA, adoption + brown-field migration, FinOps, readiness gate, enable-before-enforce.

Project

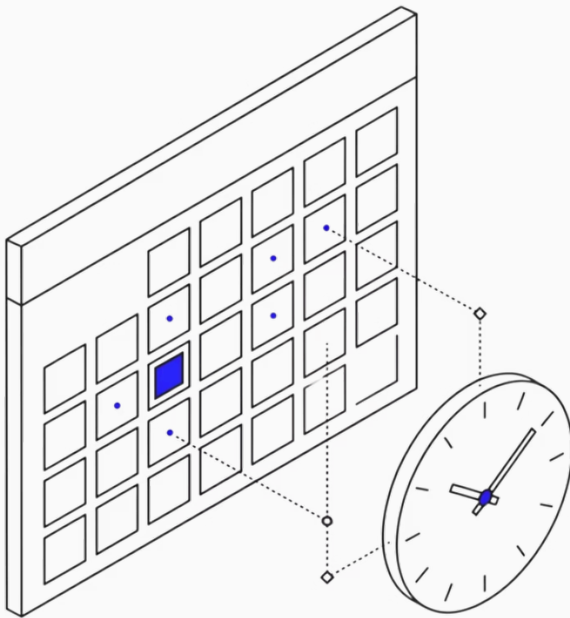
Routes all model calls through the sanctioned client; keeps domain-specific controls; migrates legacy calls.

The catalogue isn't the hard part. This climb is.

Architecture defines the bar · Engineering builds the machine · Product runs it · Governance owns the verdict — and enterprise AI fails when no one commits past Level 1.

Choose Your Cohort

One day. Three hours. Production-ready architecture you can enforce from Monday morning.



July 5, 2026

5:00 – 8:00 PM BST

July 12, 2026

9:30 AM – 12:30 PM BST

July 26, 2026

4:30 – 7:30 PM BST



\$499 USD

Save 20%+ with Maven for Teams · Backed by the Maven Guarantee

Save 20% on individual seat. Use code **AISTN**

[Maven Signup Page](#)